CODE GENERATION

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 1: Size Each Subtask So You Can Review Every Line

- Every subtask you hand to the model must produce code small enough to read line by line, typically 10 to 50 lines.
- If the output exceeds that budget, reject it and ask for a smaller scope or further decomposition.
- This is a standing constraint on every interaction, not a one-time workflow step.

Guideline 1: Reasoning

- a piece-by-piece structured workflow reduced cognitive load and improved correctness judgments
- defect detection falls sharply past 200 to 400 lines/hour; AI-generated code is harder to review than human-authored code
- as code volume grows, single-pass decisions accumulate unreviewed context and system quality degrades
- AI assistant users wrote less secure code and were significantly more likely to believe it was secure

Guideline 1: Key Practices - Prompting

- State the expected size in the prompt: "Implement only the loading function, around 20 to 30 lines, with type hints and a docstring"
- If a step would not fit in 50 lines, decompose it before asking for code, not after
- One concept per prompt: "loading and validating" is two prompts, not one

Guideline 1: Key Practices - Reviewing

- Read every line out loud or in your head and if you find yourself skipping a block "because it is obviously fine," stop and read it
- For each branch and loop, name the case it handles and the case it does not; if you cannot, you have not reviewed it
- If the model returned more than your budget, reject the output and ask for a smaller piece

Guideline 2: Ask for Plans as Options, Not as Answers

- For any task larger than a single function, ask the model for two or three plans with the trade-offs between them, not a single recommendation.
- At each step inside the chosen plan, ask it to name the meaningful alternatives and why it picked one.
- You, the developer, make the decisions. The model surfaces options and the reasons behind them.

Guideline 2: Reasoning

- plan-then-implement gives up to 25.4% relative Pass@1 improvement over direct generation
- role-based decomposition gave 29.9% to 47.1% improvement
- the human must decide at each decomposition step; errors at planning compound through every step that follows

Guideline 2: Key Practices - Multiple Plans

- Ask for two or three complete plans, not one; require the model to state what each is better and worse at
- Reject plans that differ only cosmetically; ask for plans that differ on data structure, algorithm, failure mode, or cost
- Pick one plan explicitly in the prompt before going further: "let's go with plan B" makes the decision visible

Guideline 2: Key Practices - Options and Review

- For each non-trivial step, ask the model to name the alternatives it considered and why it picked one
- Treat vague justifications ("more efficient", "idiomatic") as defaults, not decisions and push for a concrete reason tied to your problem
- If a step's choice depends on an unverifiable assumption, promote it into the prompt and decide whether it is true before continuing
- Verify the chosen plan covers all requirements, not just the happy path

Guideline 3: Ground Prompts in Your Existing Codebase

- Before asking for code that will live in an existing project, show the model what it has to fit with: signatures, type definitions, naming conventions, and the libraries already in use.
- Tell the model to prefer existing libraries and abstractions.
- When it proposes something different, require a justification against what is already in the project, not against alternatives in the abstract.

Guideline 3: Reasoning

- developers avoid AI tools most because suggestions use unfamiliar APIs they cannot understand
- providing surrounding code as context is one of the strongest levers for function-level correctness
- repository context retrieval improves code completion by more than 10% over in-file-only baselines
- specifying which libraries to use, reduces invented dependencies and duplication
- skipping context causes fragmentation: two HTTP clients, two date formatters, competing test helpers in the same project

Guideline 3: Key Practices - Context

- Paste the signatures of functions your new code will call or be called by as the model needs the interface, not just the description
- List libraries already in use and tell the model to prefer them: "This project uses requests for HTTP. Use that, not alternatives."
- Include 5 to 10 lines of an existing similar function as a style reference
- Name the conventions the project follows: snake_case vs camelCase, exception types, logging patterns

Guideline 3: Key Practices - Reviewing for Fit

- Read the imports first, before the logic as an unexpected import signals a choice you should validate
- For every new dependency, ask: is this already in the project? Is the benefit worth a new entry in the manifest?
- Check naming: does the new code use the same vocabulary as its neighbors?
- When the model suggests a different library, ask it to compare specifically against the existing one, not the field in general

Guideline 4: Drive Generation with Executable Tests

- Before asking for code, write test cases that capture the intended behavior, including edge cases.
- Hand the model the signature and the tests together, and require an implementation that passes them.
- Treat any tests the model proposes as hypotheses about your intent, not ground truth: review each one, then accept, edit, or reject it.

Guideline 4: Reasoning

- giving LLMs the problem statement plus tests consistently improved success on MBPP and HumanEval vs. problem alone
- test-driven workflow made participants significantly more likely to correctly evaluate AI-generated code and reported less cognitive load
- executable acceptance criteria improved semantic consistency between requirements and generated code
- tests pin intent in machine-checkable form: review becomes "did it pass my tests" rather than "does this prose look right"

Guideline 4: Key Practices - Prompting with Tests

- Write 3 to 6 test cases before the prompt: cover the normal case, one boundary, and at least one the model is likely to miss (empty input, wrong types, large input)
- Hand the model the signature and the tests together; ask for an implementation that passes them and explains any assumption made
- If the model writes tests for you, read each one and ask "does this match what I actually want?" before running it

Guideline 4: Key Practices - Reviewing

- Run the tests yourself as a model saying "this passes the tests" is not evidence
- Add at least one adversarial test the model did not see before accepting the code
- Watch for test-overfit: if the fix is a special case for the failing input (e.g., if x == 42: return 7), reject it

Guideline 5: Specify Preconditions, Postconditions, and Edge Cases

- Before asking for code, write down the input types and ranges, the output contract, and the behavior on empty and boundary inputs.
- Name which exceptions must be raised and when.
- Use assertive language ("must", "is required to") rather than "should" or "may".

Guideline 5: Reasoning

- four of the five most-applied prompt-improvement patterns are about specification: algorithmic details (57%), I/O format (44%), more examples (24%), postconditions (23%)
- I/O format and pre/postconditions rated as most useful patterns in practice
- Assertive language ("must") is applied less often but rated highly and practitioners find it effective when they try it
- Tests pin behavior by example; specifications pin behavior by contract and you want both

Guideline 5: Key Practices - Prompting with Specs

- State each parameter's type, shape, and allowed range; explicitly say what empty, zero, negative, and very-large inputs must do
- State the output contract with the same precision: type, shape, invariants ("sorted ascending", "sums to 1", "non-empty")
- Name the exceptions and when they fire: "Raise ValueError if x is negative" beats "handle bad inputs gracefully"
- Give two or three small input/output examples that together cover a normal case and at least one edge case

Guideline 5: Key Practices - Reviewing

- For every stated edge case, run a test that exercises exactly that input
- Confirm exception types and messages match the spec, and the code does not swallow them in a broad except block
- If the code silently accepts something you told it to reject, that is a spec violation, not a style issue

Guideline 6: Have the Model Critique Its Own Output Before You Run It

- After the model produces code, do not run it yet. Prompt the model to critique its own output against the spec: list assumptions made, edge cases not handled, and lines where it is uncertain.
- Read the critique and decide what is actually a problem.
- Then ask for a revision based on your decisions, not the model's.

Guideline 6: Reasoning

- model feedback on its own output followed by refinement beat single-pass generation across multiple tasks including code
- explicit self-reflection between attempts improved Pass@1 on HumanEval
- self-repair works only when feedback quality is high; quality drops on harder problems and evaluate the critique, do not trust it
- Critique before running catches what tests miss: assumptions made when the spec was silent, cases you forgot to test

Guideline 6: Key Practices - Prompting for Self-Critique

- After the model returns code, ask for three specific lists: assumptions made, edge cases not handled, lines the model is uncertain about
- Forbid the model from rewriting the code in the critique step as you want the audit, not a new draft
- For each item, ask one follow-up: "If this assumption is wrong, what breaks?" this tells you whether the item matters

Guideline 6: Key Practices - Reviewing the Critique

- Read every item; treat each as a hypothesis the model is offering, not a fact
- For each item, decide: is this real, a non-issue, or is the model wrong about being wrong? All three happen
- If the critique misses something you can see, the model's understanding is shallower than expected but be more skeptical of the rest
- Only after forming your own verdict should you ask for a revision; specify what to fix, not 'fix the issues you found'

References

[1] Perry et al. (2023). Do Users Write More Insecure Code with AI Assistants? CCS '23. DOI: 10.1145/3576915.3623157

[2] Sandoval et al. (2023). Lost at C: A User Study on the Security Implications of LLM Code Assistants. USENIX Security 2023.

[3] Fakhoury et al. (2024). LLM-Based Test-Driven Interactive Code Generation. IEEE TSE. DOI: 10.1109/TSE.2024.3428972

[4] Jiang et al. (2024). Self-Planning Code Generation with LLMs. ACM TOSEM. DOI: 10.1145/3672456

[5] Dong et al. (2024). Self-Collaboration Code Generation via ChatGPT. ACM TOSEM 33(7). DOI: 10.1145/3672459

[6] Lin et al. (2025). SOEN-101: Code Generation by Emulating Software Process Models. ICSE 2025. DOI: 10.1109/ICSE55347.2025.00140

[7] Zhang et al. (2025). Empowering Agile-Based Generative Software Development. ACM TOSEM. DOI: 10.1145/3702987

[8] Ishibashi and Nishimura (2024). Self-Organized Agents: A LLM Multi-Agent Framework. arXiv:2404.02183

[9] Liang et al. (2024). A Large-Scale Survey on the Usability of AI Programming Assistants. ICSE '24. DOI: 10.1145/3597503.3608128

References (cont.)

[10] Khojah et al. (2025). The Impact of Prompt Programming on Function-Level Code Generation. IEEE TSE 51(8), pp. 2381-2395.

[11] Zhang et al. (2023). RepoCoder: Repository-Level Code Completion. EMNLP 2023. DOI: 10.18653/v1/2023.emnlp-main.151

[12] Midolo et al. (2026). Guidelines to Prompt LLMs for Code Generation: An Empirical Characterization. arXiv:2601.13118

[13] Mathews and Nagappan (2024). Test-Driven Development for Code Generation. ASE 2024. DOI: 10.1145/3691620.3695527

[14] Madaan et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023. arXiv:2303.17651

[15] Shinn et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. arXiv:2303.11366

[16] Olausson et al. (2024). Is Self-Repair a Silver Bullet for Code Generation? ICLR 2024. arXiv:2306.09896